Day 115: k-Nearest Neighbours From Scratch — a Real Curse-of-Dimensionality Demonstration

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 17, Day 4 | Complete study guide

Watch: StatQuest: K-nearest neighbors, Clearly Explained | The Curse of Dimensionality (Week 03-01) (Zubair Khalid)

Script: `day115_k_nearest_neighbors.py` · every number in this guide is copied by program from `day115_run_output.txt`, the real output of that script.

k-nearest neighbours written from scratch: it matches scikit-learn exactly at k = 1 and 5, and reaches 0.9829 on handwritten digits with no training at all. Its one assumption, that 'near' means 'similar', breaks in many dimensions, and today measures how. Among 1,000 random points in 1,000 dimensions the nearest is 0.9447 of the average distance away. Adding 1,024 useless dimensions to the digits drops accuracy from 0.9829 to 0.6545, and PCA recovers much of what 256 of them cost (0.9555 against 0.8779).

0. Where today fits

DEFINITION — start here, no prior knowledge assumed

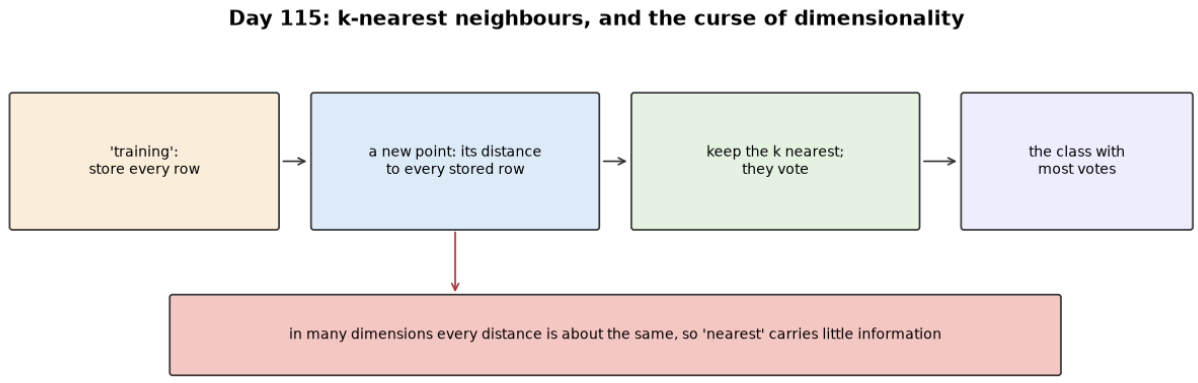
k-nearest neighbours (kNN) stores the training data and, for a new point, finds the k training points with the smallest distance to it; their majority class is the prediction. There is no training step, which is why it is called a **lazy learner**.

The **curse of dimensionality** is the set of ways high-dimensional space defies intuition: points spread out, every point is far from every other, and the nearest and the farthest become nearly the same distance away.

REAL-WORLD SCENARIO

Guessing a house's price from the five most similar houses sold nearby. That works when 'similar' is judged on a few things that matter; judged on hundreds of details, most of them irrelevant, every house looks about equally different.

SYSTEM DIAGRAM



kNN: store, measure, vote; and where it breaks.

Today's part for the week's build

`day115_exercise.py` is the vote and PCA, checked by hand and on the edge cases. Its real output:

ACTUAL OUTPUT

```
query (0.4, 0.4): k=1 -> 0, k=3 -> 0, k=5 -> 1 (by hand: 0, 0, 1)
query (5.5, 4.0): k=1 -> 1 (by hand: 1)
k = all 5 points: every query gets the majority class: [1, 1]
a 2-2 tie at x = 1.6 (k = 4): the nearest neighbour is x = 1, class 1 -> 1
PCA on spreads 3, 1, 0.1: variance shares [0.9008, 0.0983, 0.0009] (by hand about 9 : 1 : 0.01 of
    10.01)
directions are orthonormal: largest gap of V V^T from the identity 2.2e-16
```

Data: handwritten digits and breast cancer, bundled with scikit-learn, and uniform random points for the pure geometry. Day 112's scaling and splitting are imported; scikit-learn's KNeighborsClassifier is an outside check only.

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. By hand: five points

DEFINITION — start here, no prior knowledge assumed

Three points near the origin and two far away. The query (0.4, 0.4) sits among the near ones; the answer depends on k.

REAL-WORLD SCENARIO

Asking your nearest neighbour, your three nearest, or the whole street.

MATHEMATICS

$$d(x, z)^2 = \sum_j (x_j - z_j)^2 = |x|^2 + |z|^2 - 2x \cdot z$$

prediction = the most common class among the k nearest

IMPLEMENTATION

```
def knn_predict(Xtr, ytr, X, k, chunk=500):
    n_classes = int(ytr.max()) + 1; out = []
    for s in range(0, len(X), chunk):
        D = sqdist(X[s:s + chunk], Xtr)
        idx = np.argpartition(D, k - 1, axis=1)[:, :k] if k < len(ytr) else
            np.tile(np.arange(len(ytr)), (len(D), 1))
        votes = np.zeros((len(D), n_classes))
        for j in range(k):
            votes[np.arange(len(D)), ytr[idx[:, j]]] += 1
        d = np.take_along_axis(D, idx, 1)
        tie = (votes == votes.max(1, keepdims=True)).sum(1) > 1
        for r in np.where(tie)[0]:
            best = np.where(votes[r] == votes[r].max())[0]
            order = idx[r][np.argsort(d[r])]
            out_r = next(ytr[i] for i in order if ytr[i] in best); votes[r, out_r] += 0.5
        out.append(np.argmax(votes, 1))
    return np.concatenate(out)
```

ACTUAL OUTPUT

```
training: (0,0) class 0, (1,0) class 0, (0,1) class 1, (5,5) class 1, (6,5) class 1; query (0.4,
    0.4)
squared distances: [0.32, 0.52, 0.52, 42.32, 52.52]
  k = 1: class 0  (nearest is (0,0))
  k = 3: class 0  (two of the three nearest are class 0)
  k = 5: class 1  (all five vote: three are class 1)
Actual output from this lesson's run.
```

What it means

k = 1 and k = 3 follow the nearby points (class 0); k = 5 lets the far-away points outvote them (class 1). k decides how local the vote is.

WHY CHOOSE THIS?

No training, and every prediction can be explained by its neighbours.

WHY NOT THIS? (TRADEOFFS)

k changes the answer, and must be chosen.

2. Correctness

DEFINITION — start here, no prior knowledge assumed

scikit-learn's brute-force kNN must agree, except where two classes tie in votes and the two versions break ties differently.

REAL-WORLD SCENARIO

Two people counting the same raised hands should agree unless the vote is tied.

ACTUAL OUTPUT

```
digits, k =  1: the same prediction on 1.0000 of 539 test images; accuracy 0.9833 against 0.9833
digits, k =  5: the same prediction on 1.0000 of 539 test images; accuracy 0.9870 against 0.9870
digits, k = 15: the same prediction on 0.9981 of 539 test images; accuracy 0.9852 against 0.9833
ties between equally common classes are the only place two correct versions can differ
Actual output from this lesson's run.
```

What it means

Identical predictions at k = 1 and 5 (1.0000); at k = 15, 0.9981, one image of 539, decided by a tie. Here ties go to the class with the nearest neighbour; scikit-learn uses a different rule.

WHY CHOOSE THIS?

A clear check: only ties may differ.

WHY NOT THIS? (TRADEOFFS)

It checks the vote, not whether kNN suits the problem.

3. Choosing k

DEFINITION — start here, no prior knowledge assumed

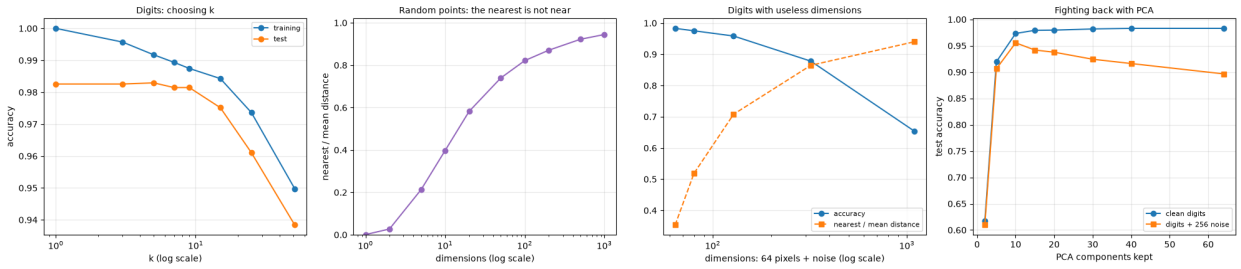
Training accuracy is meaningless at k = 1: each training image finds itself. Test accuracy shows the real trade-off.

REAL-WORLD SCENARIO

One witness is sharp but unreliable; a crowd is reliable but blurred.

ACTUAL OUTPUT

```
    k  train acc  test acc
    1     1.0000    0.9826
    3     0.9957    0.9826
    5     0.9917    0.9829
    7     0.9893    0.9814
    9     0.9874    0.9814
   15     0.9843    0.9751
   25     0.9736    0.9610
   51     0.9498    0.9384
k = 1 is perfect on training rows because each row finds itself; larger k smooths, and eventually
    blurs the classes
Actual output from this lesson's run.
```



From left: choosing k on digits; the nearest-to-mean distance ratio on random points; digits with noise dimensions; PCA.

What it means

Small k is best on digits: 0.9829 at k = 5, falling to 0.9384 at k = 51, where the vote reaches into other digits. On this data there is little noise for a larger k to average away.

WHY CHOOSE THIS?

One setting, easy to choose by cross-validation.

WHY NOT THIS? (TRADEOFFS)

The best k depends on how noisy and how dense the data is.

4. Scaling

DEFINITION — start here, no prior knowledge assumed

A distance adds up squared differences in each feature's own units. A feature measured in thousands dominates one measured in fractions, whatever their real importance.

REAL-WORLD SCENARIO

Comparing houses by price in pounds and size in rooms: a difference of one room is lost in a difference of a thousand pounds.

ACTUAL OUTPUT

```
the largest spreads: worst area 568.9, mean area 351.6, area error 45.5; the smallest: fractal dimension error
    0.0026
raw measurements: 0.9345; standardised: 0.9637
on raw data, 'worst area' alone makes up 0.6700 of the squared distance between two tumours (median over all
    pairs): one feature decides who is near
Actual output from this lesson's run.
```

What it means

On raw breast-cancer measurements, 'worst area' alone makes up 0.6700 of the typical squared distance between two tumours, so the neighbours are chosen almost by one feature. Standardising lifts accuracy from 0.9345 to 0.9637.

WHY CHOOSE THIS?

Standardising is one line and usually essential.

WHY NOT THIS? (TRADEOFFS)

It treats every feature as equally important, which is also a choice.

5. The curse on random data

DEFINITION — start here, no prior knowledge assumed

For 100 query points among 1,000 random points in a cube of d dimensions: the **relative contrast** (farthest – nearest) / nearest, and the nearest distance as a share of the mean distance.

REAL-WORLD SCENARIO

In a crowded room your nearest neighbour is much closer than the person across it; in a thousand-dimensional room, everyone is across it.

MATHEMATICS

$$\textit{relative contrast} = (d_{max} - d_{min}) / d_{min} \to 0 \text{ as } d \text{ grows, for independent random features}$$

ACTUAL OUTPUT

```
 dimensions  relative contrast  nearest / mean
          1          2520.4054          0.0009
          2            70.8257          0.0272
          5             6.8245          0.2132
         10             2.7404          0.3974
         20             1.3571          0.5819
         50             0.6810          0.7396
        100             0.4279          0.8215
        200             0.2897          0.8705
        500             0.1694          0.9222
       1000             0.1169          0.9447
as dimensions grow, the nearest point is barely nearer than the average one: 'nearest' stops
    meaning much
Actual output from this lesson's run.
```

What it means

In 2 dimensions the relative contrast is 70.8257: the farthest point is about 72 times as far away as the nearest. In 1,000 dimensions it is 0.1169: the farthest is only about 12% further than the nearest, and the nearest point is 0.9447 of the way to the average one. When every distance is about the same, the k 'nearest' are hardly nearer than any others.

WHY CHOOSE THIS?

Pure geometry: no model, so nothing can hide.

WHY NOT THIS? (TRADEOFFS)

Uniform random features are the worst case; real features are correlated and lie on fewer effective dimensions.

6. The curse in volume

DEFINITION — start here, no prior knowledge assumed

A cube holding 1% of uniformly spread points needs a side of $0.01^{1/d}$. In one dimension that is 1% of the range; in many, nearly all of it.

REAL-WORLD SCENARIO

To collect 1% of a city's residents by a small square on the map is easy; to collect 1% matching on a hundred attributes, the 'neighbourhood' is the whole city.

ACTUAL OUTPUT

```
 dimensions  edge for 1%  nearest of 1,000  cube diagonal
          1       0.0100            0.0003         1.0000
          2       0.1000            0.0141         1.4142
          5       0.3981            0.1928         2.2361
         10       0.6310            0.5098         3.1623
         20       0.7943            1.0516         4.4721
         50       0.9120            2.1132         7.0711
        100       0.9550            3.3347        10.0000
        200       0.9772            5.0405        14.1421
        500       0.9908            8.4052        22.3607
       1000       0.9954           12.1731        31.6228
in 100 dimensions the 'local' 1% cube spans 95% of every side: a neighbourhood is no longer local
Actual output from this lesson's run.
```

What it means

In 10 dimensions the 1% cube already spans 0.6310 of every side; in 100, 0.9550. The measured distance to the nearest of 1,000 points grows from 0.0141 in 2 dimensions to 3.3347 in 100, a third of the cube's diagonal. 'Local' stops existing.

WHY CHOOSE THIS?

Explains why more features can make kNN worse.

WHY NOT THIS? (TRADEOFFS)

Real data usually fills far fewer dimensions than it has columns.

7. The curse on real data

DEFINITION — start here, no prior knowledge assumed

Digits get extra columns of uniform noise on the same 0 to 16 range as a pixel. The information is unchanged; only the geometry is diluted.

REAL-WORLD SCENARIO

Judging handwriting with a page of random scribbles attached to every sample.

ACTUAL OUTPUT

```
each added dimension is uniform noise on 0..16, the same range as a pixel
  added  dimensions  accuracy  nearest / mean distance
      0          64    0.9829                    0.3535
     16          80    0.9751                    0.5187
     64         128    0.9584                    0.7080
    256         320    0.8779                    0.8647
   1024        1088    0.6545                    0.9398
Actual output from this lesson's run.
```

What it means

Accuracy falls from 0.9829 to 0.8779 with 256 noise columns and 0.6545 with 1,024, while the nearest-to-mean ratio climbs from 0.3535 to 0.9398: the same concentration as random points. Trees suffer much less, because they choose which columns to ask about (Day 111's useless-column test); kNN cannot choose, because every column enters every distance.

WHY CHOOSE THIS?

A direct, real measure of the harm.

WHY NOT THIS? (TRADEOFFS)

Added noise is the extreme case; real irrelevant features are rarely this loud.

8. Fighting back with PCA, and the cost of a lazy model

DEFINITION — start here, no prior knowledge assumed

PCA (principal component analysis) finds the directions in which the data varies most. Keeping only the first few reduces the dimensions. It is fitted on training rows only.

REAL-WORLD SCENARIO

Describing faces by the few measurements that vary most between people, instead of by every pixel.

MATHEMATICS

$X - mean = U \Sigma V^T$ *(singular value decomposition); the rows of* V^T *are the directions,* Σ^2 *their variances*

IMPLEMENTATION

```
def pca_fit(X, k):
    mu = X.mean(0); _, s, Vt = np.linalg.svd(X - mu, full_matrices=False)
    return mu, Vt[:k], (s ** 2) / (s ** 2).sum()
```

ACTUAL OUTPUT

```
digits: the first 10 PCA directions hold 0.7382 of the variance; the first 30, 0.9591
 components  clean digits  digits + 256 noise
          2        0.6171              0.6096
          5        0.9195              0.9061
         10        0.9733              0.9555
         15        0.9792              0.9417
         20        0.9796              0.9377
         30        0.9818              0.9243
         40        0.9829              0.9161
         64        0.9829              0.8965
PCA keeps the directions with most variance: a noise dimension has variance 21.33, an average pixel 18.77 and
    the busiest pixel 42.72,
  so noise is not simply ranked last, and only part of it is filtered out
    250 training images: predicting 539 test images takes 7.3 ms (mean of 5, within this run)
    500 training images: predicting 539 test images takes 15.7 ms (mean of 5, within this run)
   1000 training images: predicting 539 test images takes 23.7 ms (mean of 5, within this run)
   1257 training images: predicting 539 test images takes 26.5 ms (mean of 5, within this run)
Actual output from this lesson's run.
```

What it means

On clean digits, 10 components give 0.9733 and 40 match all 64 (0.9829). On noisy digits, 10 components give 0.9555 against 0.8965 for all of them: PCA throws much of the noise away. It cannot throw away all of it, because a noise column has variance 21.33, above an average pixel's 18.77: PCA keeps what varies, not what matters. And kNN pays at prediction time: the cost grows with the training set, from 7.3 to 26.5 ms here, because every prediction compares against every stored row.

WHY CHOOSE THIS?

Fewer dimensions: better distances and faster predictions.

WHY NOT THIS? (TRADEOFFS)

PCA is unsupervised: it can discard a low-variance direction that separates the classes.

Interview preparation — technical questions

Q: How does kNN predict?

A: Distances to every training point, the k smallest, and a majority vote.

Q: Why must features be scaled for kNN?

A: Distances add up raw units: unscaled, one feature made up 0.6700 of the typical distance.

Q: How do you choose k?

A: Cross-validation; here k = 5 was best on digits, k = 51 cost 4 points.

Q: Why is kNN slow at prediction?

A: It compares each new point with every stored row; cost grows with the training set.

Q: How do you compute all distances efficiently?

A: |x|^2 + |z|^2 - 2x.z, one matrix product for every pair.

Interview preparation — theoretical questions

Q: What is the curse of dimensionality?

A: In many dimensions distances concentrate: the nearest of 1,000 random points was 0.9447 of the mean distance in 1,000 dimensions.

Q: Why does kNN suffer more than trees?

A: Every feature enters every distance; trees choose which features to ask about.

Q: How do you fight it?

A: Fewer, better features: selection, PCA (0.9555 against 0.8779 here), or a learned distance.

Q: Why does PCA not remove all the noise?

A: It keeps directions of high variance, and the noise had more variance (21.33) than an average pixel (18.77).

Key takeaway and what's next

> **KEY TAKEAWAY**
> kNN is the simplest model in the course and, on clean low-dimensional data such as these digits, one of the most accurate. Everything rests on distance, so it fails exactly where distance fails: unscaled features, and many irrelevant dimensions, where every point becomes about equally far from every other.

- Digits: 0.9829 at k = 5, matching scikit-learn exactly.
- Breast cancer: raw 0.9345, standardised 0.9637.
- Random points, 1,000 dimensions: nearest / mean distance 0.9447.
- Digits + 1,024 noise dimensions: 0.6545; + 256 with 10 PCA components: 0.9555.

Suggested watch order, if starting from zero

- 1. StatQuest: K-nearest neighbors, Clearly Explained — the method in a few minutes.
- 2. The Curse of Dimensionality (Week 03-01) (Zubair Khalid) — a lecture on why high dimensions break nearness.

Day 116 preview

- **Stacking and blending**: training a model to combine other models' predictions, with a real meta-model demonstration.